EM-KalmanNet: AI-Aided Kalman Tracking in Partially Known Time-Varying State-Space Models

Ori Cohen, Xiaoyong Ni, Nir Shlezinger, and Tirza Routtenberg

***Abstract*—Smart transportation systems rely heavily on accurate state tracking for tasks such as autonomous vehicle navigation, traffic monitoring, and cooperative sensing, often operating in environments with partially known and time-varying dynamics. Classical model-based approaches, such as the expectation-maximization (EM) Kalman filter, require accurate dynamical models and thus may degrade under the complex and non-stationary conditions encountered in real-world transportation scenarios. Recent AI-aided tracking methods, including KalmanNet and its extensions, improve robustness to model mismatch, yet they typically struggle to adapt online to evolving dynamics without labeled data. In this work, we propose *EM-KalmanNet*, an AI-aided tracking algorithm that integrates EM with KalmanNet through deep unfolding. Our design introduces an unfolded EM-based architecture in which the E-step is implemented via an enhanced RTSNet smoother, while the M-step is rendered differentiable, yielding a compact, interpretable, and trainable machine learning model. We further develop a dedicated training methodology for robust operation under partially specified state-space models with block-wise varying dynamics, representative of changing transportation environments. Numerical results demonstrate that EM-KalmanNet enables accurate tracking under complex time-varying dynamics, making it a promising framework for adaptive state estimation in smart transportation systems.**

***Index Terms*—Kalman filter, deep learning, EM algorithm.**

I. Introduction

Tracking of dynamic systems is a fundamental task in signal processing [1], with notable applications in smart transportation and localization [2]. In practice, such systems can be highly complex and non-stationary, i.e., their statistical behavior is difficult to fully characterize and may vary over time [3]. These challenges make reliable tracking non-trivial, particularly when the underlying system evolves in ways that are only partially known.

Conventional tracking approaches are predominantly based on the Kalman filter (KF) [4] and its variants [5]–[7]. These methods rely on the explicit formulation of a state space (SS) model [8] and can be readily applied in time-varying scenarios when the variations in the model are fully known. When some of the model parameters vary in an unknown fashion, they are often inferred via system identification methods [9], with the expectation-maximization (EM) algorithm [10] being widely adopted for this purpose [11], [12]. However, model-based methods require accurate characterization of the dynamics, typically assuming linear and Gaussian SS models, and they struggle when such assumptions do not hold. Furthermore, the integration of EM into Kalman-based pipelines introduces computational overhead, which can limit applicability in rapidly varying systems.

This work was support by the Israel Science Foundation (ISF) under grant no. 3314/25. O. Cohen, N. Shlezinger, and T. Routtenberg are with the ECE School, Ben-Gurion University of the Negev, Israel (e-mail: orico@post.bgu.ac.il; {nirshl; tirzar}@bgu.ac.il). X. Ni is with the Neuro-X institute of EPFL, Geneva, Switzerland (e-mail: xiaoyong.ni@epfl.ch).

Recently, there has been growing interest in artificial intelligence (AI)-aided KFs for enabling interpretable tracking in partially known SS models [13]. Various methodologies combine deep neural networks (DNNs) with KFs for tracking [14]–[16] and system identification [17], [18]. Specifically, the KalmanNet method [19] and its variants [20]–[23] have been shown to extend the applicability of KFs to complex and partially specified dynamics, where model-based techniques fail. Nonetheless, as their operation fundamentally depends on training data, the performance of these approaches degrades when the statistical model deviates from the training distribution. To mitigate this issue, the work [24] incorporated hypernetworks that enable adaptation to a limited range of variations without full retraining, while [25] proposed meta-learning tools to facilitate rapid retraining from supervised data. However, these existing methods neither indicate when adaptation is required nor provide the full flexibility for AI-aided KFs to operate reliably in complex time-varying SS models without access to online labeled data.

In this work, we propose *EM-KalmanNet*, an AI-aided tracking methodology that fuses EM principles [10] with the KalmanNet framework [19]. Our approach converts the EM-KF into a discriminative machine learning model [26] by integrating KalmanNet with deep unfolding techniques [27]. EM-KalmanNet is designed to learn to operate and adapt in partially known and complex SS models, particularly in the challenging setup of time-varying state evolution. To this end, we unfold the EM-KF with a small, predefined number of passes, and leverage the RTSNet architecture, which extends KalmanNet to smoothing tasks [20], as a core building block in the overall pipeline. We further introduce a dedicated learning algorithm tailored for this unfolded structure, enabling robust adaptation to varying dynamics. Our experimental study systematically demonstrates the superiority of EM-KalmanNet in tracking complex and time-varying systems.

The rest of this paper is organized as follows: Section II formulates the problem. EM-KalmanNet is detailed in Section III, and evaluated in Section IV. Section V provides concluding remarks.

II. System Model and Preliminaries

A. System Model

Signal Model: We consider a dynamic system in discrete-time with hidden state $\boldsymbol{x}_t \in \mathbb{R}^m$ that is related to an observation $\boldsymbol{y}_t \in \mathbb{R}^n$ according to a SS model of the form

$$\boldsymbol{x}_t = f_t(\boldsymbol{x}_{t-1}) + \boldsymbol{w}_t, \quad \boldsymbol{y}_t = h(\boldsymbol{x}_t) + \boldsymbol{v}_t. \tag{1}$$

In (1), $f_t(\cdot)$ is the state evolution function, and $h(\cdot)$ is the stationary observation function, where $\boldsymbol{w}_t$ and $\boldsymbol{v}_t$ are temporally independent noise signals. In particular, it is assumed that the

state evolution $f_t(\cdot)$ changes in time in a *block-wise* fashion, i.e., that the function changes every T time instances.

Problem Formulation: We focus on *block-wise smoothing*, where at block index i, $\boldsymbol{X}_i = \{\boldsymbol{x}_\tau\}_{\tau=(i-1)\cdot T+1}^{i\cdot T}$ is estimated from the observed $\boldsymbol{Y}_i = \{\boldsymbol{y}_\tau\}_{\tau=(i-1)\cdot T+1}^{i\cdot T}$. We particularly focus on smoothing subject to the following challenges:

- **C1** The observation function $h(\cdot)$ may be a crude approximation of the true underlying model.
- **C2** The distribution of the noise signals is unknown and may be non-Gaussian.
- **C3** The variations in $f_t(\cdot)$ are unknown.
- **C4** The estimate of $\boldsymbol{X}_i$ must be produced with minimal latency.

To cope with C1-C4, we assume that during design, one has access to data from the underlying SS model. Such data is given by

$$\mathcal{D} = \left\{ \{(\boldsymbol{x}_{j,t}, \boldsymbol{y}_{j,t})\}_{t=1}^{T}, \right\}_{j=1}^{|\mathcal{D}|}. \tag{2}$$

B. Preliminaries

Kalman Smoothing: The classic approach for smoothing is based on extensions of the KF, such as the extended Rauch-Tung-Striebel (RTS) algorithm [28]. Such methods are designed for dynamics that are captured by a known SS model (violating Challenges C1 and C3) with Gaussian state and observation noise covariances $\boldsymbol{Q}$ and $\boldsymbol{R}$, respectively (violating C2).

The RTS smoother recovers the latent state variables using two linear recursive steps referred to as the forward and backward passes. The forward pass is a standard KF, which updates its *prior* for state and covariance based on past observations. For each t the forward pass computes the predicted first-order moments via

$$\hat{\boldsymbol{x}}_{t|t-1} = f_t\left(\hat{\boldsymbol{x}}_{t-1|t-1}\right), \quad \hat{\boldsymbol{y}}_{t|t-1} = h_t\left(\hat{\boldsymbol{x}}_{t|t-1}\right), \tag{3}$$

and the predicted second-order moments via

$$\hat{\boldsymbol{P}}_{t|t-1} = \hat{\boldsymbol{F}}_t \hat{\boldsymbol{P}}_{t-1|t-1} \hat{\boldsymbol{F}}_t^\top + \boldsymbol{Q}, \;\; \hat{\boldsymbol{S}}_t = \hat{\boldsymbol{H}}_t \hat{\boldsymbol{P}}_{t|t-1} \hat{\boldsymbol{H}}_t^\top + \boldsymbol{R},$$

where $\hat{\boldsymbol{F}}_t$ and $\hat{\boldsymbol{H}}_t$ are the local linearizations of $f_t(\cdot)$ and $h(\cdot)$, respectively. The predictions are updated via

$$\hat{\boldsymbol{x}}_{t|t} = \hat{\boldsymbol{x}}_{t|t-1} + \boldsymbol{\mathcal{K}}_t(\boldsymbol{y}_t - \hat{\boldsymbol{y}}_{t|t-1}), \; \hat{\boldsymbol{P}}_{t|t} = \hat{\boldsymbol{P}}_{t|t-1} - \boldsymbol{\mathcal{K}}_t \hat{\boldsymbol{S}}_t \boldsymbol{\mathcal{K}}_t^\top, \tag{4}$$

with *forward* Kalman gain (KG) $\boldsymbol{\mathcal{K}}_t = \hat{\boldsymbol{P}}_{t|t-1} \hat{\boldsymbol{H}}_t^\top \hat{\boldsymbol{S}}_t^{-1}$.

The backward pass refines the forward estimates by incorporating information from future observations, applied sequentially from $t = T-1$ to $t = 1$. For each t, the state is updated via

$$\hat{\boldsymbol{x}}_t = \hat{\boldsymbol{x}}_{t|t} + \boldsymbol{\mathcal{G}}_t\left(\hat{\boldsymbol{x}}_{t+1} - \hat{\boldsymbol{x}}_{t+1|t}\right), \tag{5}$$

while the second-order moments are updated via $\hat{\boldsymbol{P}}_t = \hat{\boldsymbol{P}}_{t|t} + \boldsymbol{\mathcal{G}}_t(\hat{\boldsymbol{P}}_{t+1} - \hat{\boldsymbol{P}}_{t+1|t})\boldsymbol{\mathcal{G}}_t^\top$. Here, the backward KG is given by $\boldsymbol{\mathcal{G}}_t = \hat{\boldsymbol{P}}_{t|t} \hat{\boldsymbol{F}}_t^\top \hat{\boldsymbol{P}}_{t+1|t}^{-1}$.

EM-KF: When smoothing in a linear Gaussian SS model (violating Challenges C1-C2), where all parameters are known except for the state-evolution function $f_t(\boldsymbol{x}) \equiv \boldsymbol{F}_t \boldsymbol{x}$, one can employ the EM algorithm to jointly estimate $\boldsymbol{F}$ together with the latent states [12]. The method starts with some initial estimate $\boldsymbol{F}^{(0)}$, and alternates between two steps: (i) The *E-step* applies the RTS smoother. At iteration l, it assumes a linear Gaussian SS model with state evolution $\boldsymbol{F}^{(l-1)}$, and computes $\{\hat{\boldsymbol{x}}_t^{(l)}, \hat{\boldsymbol{P}}_t^{(l)}, \hat{\boldsymbol{P}}_{t,t-1}^{(l)}\}_{t=1}^T$, where

$$\hat{\boldsymbol{P}}_{t,t-1}^{(l)} = \mathrm{Cov}(\boldsymbol{x}_t, \boldsymbol{x}_{t-1} | \boldsymbol{y}_1, \ldots, \boldsymbol{y}_T; \boldsymbol{F}^{(l-1)}),$$

is the lag-one correlation; (ii) The *M-step* updates $F^{(l)}$ via likelihood maximization [29]. The parameters by maximizing the expected log-likelihood, estimating the state-transition as

$$\boldsymbol{F}^{(l)}\left(\{\hat{\boldsymbol{x}}_t^{(l)}, \hat{\boldsymbol{P}}_t^{(l)}, \hat{\boldsymbol{P}}_{t,t-1}^{(l)}\}_{t=1}^T\right) = \left(\sum_{t=1}^{T} \hat{\boldsymbol{x}}_t^{(l)} (\hat{\boldsymbol{x}}_{t-1}^{(l)})^\top + \hat{\boldsymbol{P}}_{t,t-1}^{(l)}\right) \times \left(\epsilon \cdot \boldsymbol{I}_m + \sum_{t=1}^{T} \hat{\boldsymbol{x}}_{t-1}^{(l)} (\hat{\boldsymbol{x}}_{t-1}^{(l)})^\top + \hat{\boldsymbol{P}}_{t-1}^{(l)}\right)^{-1}, \tag{6}$$

where $\epsilon > 0$ is a small constant and $\boldsymbol{I}_m$ is the $m \times m$ identity matrix. This procedure monotonically increases the (regularized) likelihood until convergence, but typically requires multiple forward–backward passes (violating Challenge C4).

RTSNet: The *RTSNet* algorithm proposed in [20] is an AI-aided RTS smoother based on the KalmanNet methodology [19]. It replaces the computation of the forward and backward gains, $\boldsymbol{\mathcal{K}}_t$ and $\boldsymbol{\mathcal{G}}_t$ in (4)-(5), with dedicated recurrent neural networks (RNNs) trained jointly in an end-to-end manner, without explicitly tracking the second-order moments. The forward pass follows the (extended) KF and the backward pass applies the update of (5), while the learned gain computations enable to cope with approximated SS models (Challenge C1) and non-Gaussian dynamics (Challenge C2). However, as the learned gains are tied to a fixed training distribution, RTSNet cannot readily accommodate time-varying SS models (Challenge C3).

III. EM-KalmanNet

This section introduces the EM-KalmanNet algorithm, illustrated in Fig. 1. It is designed to cope with block-wise variations in the state evolution (C3) by preserving the flow of the EM-KF, while (i) augmenting the *E-step* with RTSNet for tackling C1-C2; (ii) unfolding to a fixed a small number of iterations to tackle C4. Beyond combining EM-KF with RTSNet and deep unfolding, we formalize a surrogate SS model, extend RTSNet (Subsection III-A), and design a dedicated training procedure (Subsection III-B).

A. EM-KalmanNet Algorithm

We next detail the construction of EM-KalmanNet, where for simplicity, we omit the block index i, and focus on a single block.

Surrogate SS Model: The EM-KF as formulated in Subsection II-B requires a specific form of SS model, with a linear state evolution $\boldsymbol{F}$ function and known noise covariances $\boldsymbol{Q}, \boldsymbol{R}$. While this SS model may deviate from the dynamics assumed in (1), we leverage the established ability of RTSNet to deal with mismatched SS model [20]. Therefore, while the algorithm is designed to operate on data arising from (1), we utilize the parameters of a *surrogate simplified SS model*, given by

$$\boldsymbol{x}_t = \boldsymbol{F}\boldsymbol{x}_{t-1} + \tilde{\boldsymbol{w}}_t, \quad \boldsymbol{y}_t = h(\boldsymbol{x}_t) + \tilde{\boldsymbol{v}}_t, \tag{7}$$

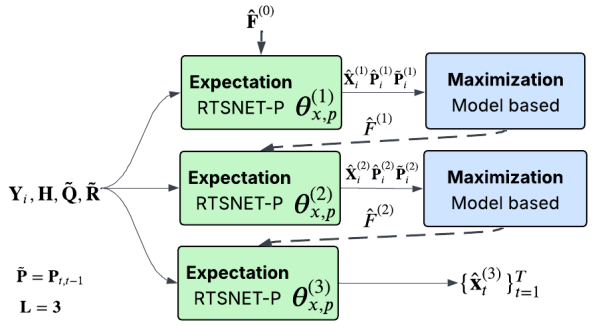

Fig. 1: EM-KalmanNet with $L = 3$ iterations illustration.

with $\tilde{\boldsymbol{Q}} = \mathrm{Cov}(\tilde{\boldsymbol{w}}_t)$ and $\tilde{\boldsymbol{R}} = \mathrm{Cov}(\tilde{\boldsymbol{v}}_t)$ (estimated in training).

RTSNetP: A key aspect of the EM-KF is that its *E-step* should track the first- and second-order state moments for different estimates of $\boldsymbol{F}$. However, RTSNet as proposed in [20] is trained for a single state evolution model, and does not track second-order moments. Thus, to have the *E-step* based on KalmanNet methodology, we propose *RTSNetP*, which extends RTSNet via: (i) To enable the RNNs, which provide the forward and backward gains in RTSNet to also account for $\boldsymbol{F}$ settings, we provide an embedding of $\boldsymbol{F}$ (obtained with a compact DNN) as another input; (ii) To provide an estimate of $\hat{\boldsymbol{P}}_t$, we use an additional lightweight DNN that integrates GRU with fully connected layers applied to the backward-gain $(\{\boldsymbol{\mathcal{G}}_t\}_{t=1}^{T-1})$ and the forward-gain RNN features used to compute $\mathbf{K}_t$. The output layer ensures that $\hat{\boldsymbol{P}}_t$ is a positive semi-definite matrix via eigenvalue clipping; (iii) The estimate of $\hat{\boldsymbol{P}}_{t,t-1}$ is obtained from the parameters of the surrogate SS model (7), the learned KG, and the estimated $\hat{\boldsymbol{P}}_t$ via the model-based formulation used in the RTS algorithm for linear Gaussian SS models, i.e., via the recursion

$$\hat{\boldsymbol{P}}_{T,T-1} = (\boldsymbol{I}_m - \boldsymbol{\mathcal{K}}_T \hat{\boldsymbol{H}}_T)\, \boldsymbol{F}\, \hat{\boldsymbol{P}}_{T-1|T-1}, \tag{8a}$$

$$\hat{\boldsymbol{P}}_{t,t-1} = \hat{\boldsymbol{P}}_{t|t}\, \hat{\boldsymbol{\mathcal{G}}}_{t-1}^{\top} + \hat{\boldsymbol{\mathcal{G}}}_t \big(\hat{\boldsymbol{P}}_{t+1,t} - \boldsymbol{F}\, \hat{\boldsymbol{P}}_{t|t}\big) \hat{\boldsymbol{\mathcal{G}}}_{t-1}^{\top}. \tag{8b}$$

Unfolded Algorithm: EM-KalmanNet is obtained by unfolding the EM-KF based on the surrogate SS model (7) into L EM iterations, where L is set to be a small integer holding **C4** (e.g., $L = 3$ as used in Section IV). The *E-step* in each iteration is implemented using a different RTSNetP model, with $\boldsymbol{\theta}_l$ denoting the parameters of RTSNetP at the lth EM iteration. The overall EM-KalmanNet operation is summarized as Algorithm 1. While Algorithm 1 is formulated for smoothing over a single block, when operating over multiple blocks as formulated in Subsection II-A, the initial value of $\boldsymbol{F}^{(0)}$ used in the ith block is set to the value of $\boldsymbol{F}$ tuned by EM-KalmanNet at the previous block.

B. Training

Algorithm 1 requires setting the parameters of RTSNetP $\{\boldsymbol{\theta}_l\}_{l=1}^L$, and surrogate covariances $\tilde{\boldsymbol{Q}}, \tilde{\boldsymbol{R}}$. These are obtained from the dataset $\mathcal{D}$ via three stages of training:

Algorithm 1: EM-KalmanNet at one block

Init : Trained $\{\boldsymbol{\theta}_l\}_{l=1}^L$; Surrogate covariances $\tilde{\boldsymbol{Q}}, \tilde{\boldsymbol{R}}$
Input : Observation $\{\boldsymbol{y}_t\}_{t=1}^T$; Initial $\boldsymbol{F}^{(0)}$
for $l = 1, \ldots, L$ **do**
  E-Step:
  $\{\hat{\boldsymbol{x}}_t^{(l)}, \hat{\boldsymbol{P}}_t^{(l)}\}_{t=1}^T \leftarrow \mathrm{RTSNetP}(\{\boldsymbol{y}_t\}, \boldsymbol{F}^{(l-1)}; \boldsymbol{\theta}_l)$;
  Compute $\{\hat{\boldsymbol{P}}_{t,t-1}^{(l)}\}_{t=1}^T$ using (8) and $\tilde{\boldsymbol{Q}}, \tilde{\boldsymbol{R}}$;
  M-Step:
  Set $\boldsymbol{F}^{(l)}$ from $\{\hat{\boldsymbol{x}}_t^{(l)}, \hat{\boldsymbol{P}}_t^{(l)}, \hat{\boldsymbol{P}}_{t,t-1}^{(l)}\}_{t=1}^T$ using (6);
return $\{\hat{\boldsymbol{x}}_t^{(L)}\}_{t=1}^T$

Stage 1: Surrogate SS Model The first step sets the parameters of the surrogate SS model in (7). While the training and test data are generated by different evolution matrices, the noise covariances remain fixed and identical in both cases. Therefore, for each trajectory of index j in $\mathcal{D}$, one can estimate $\tilde{\boldsymbol{Q}}$ and $\tilde{\boldsymbol{R}}$ via standard covariance estimation from the sequences $\{\boldsymbol{x}_{j,t} - \boldsymbol{F}_j \boldsymbol{x}_{j,t-1}\}$ and $\{\boldsymbol{y}_{j,t} - h(\boldsymbol{x}_{j,t})\}$, respectively.

Stage 2: Pre-Train RTSNetP. RTSNetP consists of two components: an RTSNet model (with parameters $\boldsymbol{\theta}_x$) for state estimation, and a DNN (with parameters $\boldsymbol{\theta}_p$) that estimates $\hat{\boldsymbol{P}}_t$ from the RTSNet output. In the first stage, we pre-train these components separately: we first train $\boldsymbol{\theta}_x$ using the loss

$$\mathcal{L}_{\mathcal{D}}^{(1)}(\boldsymbol{\theta}_x) = \frac{1}{|\mathcal{D}|T} \sum_{j=1}^{|\mathcal{D}|} \sum_{t=1}^{T} \left\| \hat{\boldsymbol{x}}_t\left(\{\boldsymbol{y}_{j,t}\}, \boldsymbol{F}_0; \boldsymbol{\theta}_x\right) - \boldsymbol{x}_{j,t} \right\|_2^2,$$

As in [30], we train the covariance DNN with parameters $\boldsymbol{\theta}_p$ via

$$\mathcal{L}_{\mathcal{D}}^{(2)}(\boldsymbol{\theta}_p) = \frac{1}{|\mathcal{D}|T} \sum_{j=1}^{|\mathcal{D}|} \sum_{t=1}^{T} \left\| \hat{\boldsymbol{P}}_t\left(\{\boldsymbol{y}_{j,t}\}, \boldsymbol{F}_0; \boldsymbol{\theta}_p\right) - \boldsymbol{e}_{j,t} \boldsymbol{e}_{j,t}^{\top} \right\|_F^2,$$

where $\boldsymbol{e}_{j,t} \triangleq \hat{\boldsymbol{x}}_t\left(\{\boldsymbol{y}_{j,t}\}, \boldsymbol{F}_0; \boldsymbol{\theta}_x\right) - \boldsymbol{x}_{j,t}$. The DNNs are provided with a fixed (and possibly mismatched) state evolution $\boldsymbol{F}_0$.

Stage 3: End-to-End Tuning. We next treat the overall augmented EM algorithm as a discriminative machine learning model [26], where each RTSNetP module is initialized with the pre-trained $\boldsymbol{\theta}_l = \{\boldsymbol{\theta}_x, \boldsymbol{\theta}_p\}$. Specifically, we leverage (i) the differentiability of the *M-Step* in (6) to backpropagate between iterations; and (ii) the interpretability of the unfolded architecture to evaluate its performance based on both its output and its intermediate state estimates. Thus, by defining $\bar{\boldsymbol{\theta}} = \{\boldsymbol{\theta}_l\}_{l=1}^L$, we use a weighted unfolded loss, with the empirical risk

$$\mathcal{L}_{\mathcal{D}}^{\mathrm{E2E}}(\bar{\boldsymbol{\theta}}) = \frac{1}{|\mathcal{D}|T} \sum_{j=1}^{|\mathcal{D}|} \sum_{t=1}^{T} \sum_{l=1}^{L} a_l \left\| \hat{\boldsymbol{x}}_t^{(l)}\left(\{\boldsymbol{y}_{j,t}\}, \boldsymbol{F}_j^{(l-1)}; \boldsymbol{\theta}_l\right) - \boldsymbol{x}_{j,t} \right\|_2^2,$$

where $a_l > 0$ are predefined weights satisfying $\sum_{l=1}^L a_l = 1$.

C. Discussion

EM-KalmanNet provides a principled framework for tracking in partially known and time-varying SS models. Its design leverages RTSNet modules to overcome model mismatches (addressing **C1**–**C2**), unfolds the EM iterations to allow rapid

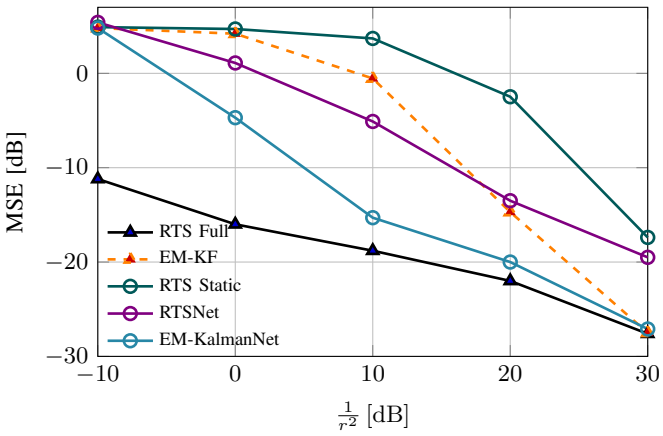

Fig. 2: MSE vs SNR, linear Gaussian SS model

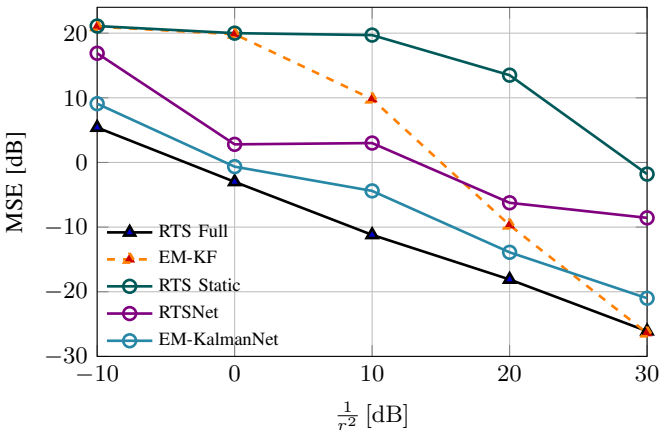

Fig. 3: MSE vs SNR, linear non-Gaussian SS model

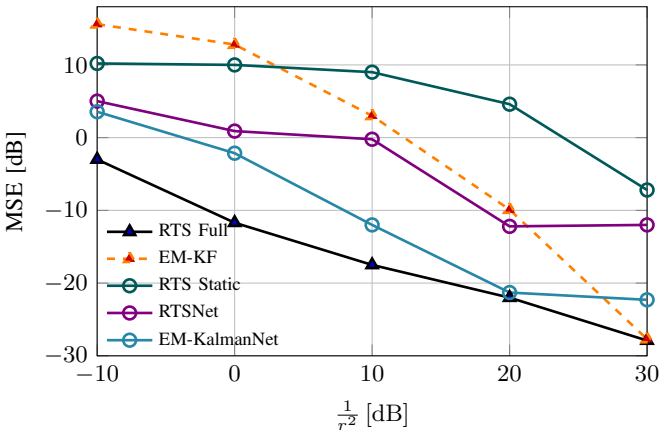

Fig. 4: MSE vs SNR, nonlinear SS model

adaptation (coping with **C4**), and integrates a surrogate SS model with a dedicated training to handle unknown variations (**C3**).

Beyond its current scope, EM-KalmanNet opens several research directions. Its methodology can be extended to accommodate variations in noise statistics and observation models, broadening its robustness in practical scenarios. Furthermore, integration with hypernetworks [24] offers the ability to dynamically generate parameters that adapt to diverse forms of dynamic variations. Such extensions, which can further enhance the flexibility and generalization of EM-KalmanNet, are left for future study.

IV. Numerical Study

We next evaluate EM-KalmanNet in SS models[1] as in (1), with $m = n = 2$, with a linear observation model. We focus on synthetic experiments to allow controlled evaluation under varying SNR and model mismatch conditions.

Time-variations are obtained by rotating $\boldsymbol{F}_0 = \begin{bmatrix} 0.83 & 0.20 \\ 0.20 & 0.83 \end{bmatrix}$ by a 0.2 radians rotation matrix every block of $T = 30$ samples.

The initial matrix $F^{(0)}$ is set using the nominal dynamics available in the training setup. This choice is made for simplicity, as the training data is generated using known dynamics. Alternatively, $F^{(0)}$ can be obtained via a least-squares estimate from the training data, yielding similar performance.

The noise covariances are $\boldsymbol{Q} = q^2\boldsymbol{I}_m$ and $\boldsymbol{R} = r^2\boldsymbol{I}_n$; in all experiments we control signal-to-noise ratio (SNR) by varying r^2 while keeping q^2 fixed at 0.01. We compare EM-KalmanNet to the model-based EM-KF, both initialized with $\boldsymbol{F}_0$. We also evaluate RTSNet operating with $\boldsymbol{F}_0$ (representing existing static data-driven approaches), as well as the model-based RTS smoother, using either only $\boldsymbol{F}_0$ (termed *RTS static*), or with full knowledge of the SS model at each block (termed *RTS Full*). All data-driven methods train with the same dataset comprised of $|\mathcal{D}| = 100$ trajectories, each including three blocks of length T.

We first use a linear observation model with $\boldsymbol{H} = \begin{bmatrix} 1 & 1 \\ 0.25 & 1 \end{bmatrix}$, using Gaussian (Fig. 2) and exponentially distributed noises (Fig. 3). We observe that in linear Gaussian models, *EM–KalmanNet* dominates across the SNR range and closely tracks the optimal RTS with the true $\boldsymbol{F}$. In non-Gaussian settings the advantage of *EM–KalmanNet* becomes larger, consistently outperforming all benchmarks. The model-based EM-KF, suffers a large gap at low/mid SNR, and manages to cope with non-Gaussianity only when the noise is weak (high SNRs). To further illustrate the gains of EM-KalmanNet, we consider a nonlinear SS model, with $h(\boldsymbol{x}) = \boldsymbol{H}\boldsymbol{x} + 0.3h_0(\boldsymbol{x})$, where $h_0(\boldsymbol{x})$ represents the transformation to polar coordinates. As observed in Fig. 4, *EM–KalmanNet* markedly outperforms EM-KF for low–to–moderate SNRs (from -10 to 20 dB).

V. Conclusion

We introduced EM-KalmanNet, which integrates EM methodology with KalmanNet. By augmenting the EM-KF with an unfolded architecture and enhanced RTSNet modules, EM-KalmanNet is designed to handle partially known and time-varying SS models. We proposed a training procedure that leverages surrogate models and end-to-end optimization to achieve robust operation in complex environments. EM-KalmanNet is shown to accurately track in challenging time-varying scenarios.

[1]Our source code and all hyperparameters can be found at https://github.com/oricoohen/EMKF_NET.git

REFERENCES

[1] S. Särkkä and L. Svensson, *Bayesian filtering and smoothing*. Cambridge university press, 2023, vol. 17.

[2] Z. Karami and R. Kashef, "Smart transportation planning: Data, models, and algorithms," *Transportation Engineering*, vol. 2, p. 100013, 2020.

[3] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with applications to tracking and navigation: theory algorithms and software*. John Wiley & Sons, 2004.

[4] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[5] S. F. Schmidt, "The Kalman filter-its recognition and development for aerospace applications," *Journal of Guidance and Control*, vol. 4, no. 1, pp. 4–7, 1981.

[6] S. J. Julier and J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, 2004.

[7] I. Arasaratnam and S. Haykin, "Cubature Kalman filters," *IEEE Trans. Autom. Control*, vol. 54, pp. 1254–1269, 2009.

[8] J. Durbin and S. J. Koopman, *Time series analysis by state space methods*. Oxford University Press, 2012.

[9] R. Mehra, "On the identification of variances and adaptive Kalman filtering," *IEEE Trans. Autom. Control*, vol. 15, no. 2, pp. 175–184, 1970.

[10] T. K. Moon, "The expectation-maximization algorithm," *IEEE Signal Process. Mag.*, vol. 13, no. 6, pp. 47–60, 1996.

[11] S. Gannot and A. Yeredor, "The Kalman filter," *Springer Handbook of Speech Processing*, pp. 135–160, 2008.

[12] R. H. Shumway and D. S. Stoffer, "An approach to time series smoothing and forecasting using the EM algorithm," *Journal of Time Series Analysis*, vol. 3, pp. 253–264, 1982.

[13] N. Shlezinger *et al.*, "Artificial intelligence-aided Kalman filters: AI-augmented designs for Kalman-type algorithms," *IEEE Signal Process. Mag.*, vol. 42, no. 3, pp. 52–76, 2025.

[14] A. Ghosh, A. Honoré, and S. Chatterjee, "DANSE: Data-driven non-linear state estimation of model-free process in unsupervised learning setup," *IEEE Trans. Signal Process.*, vol. 72, pp. 1824–1838, 2024.

[15] V. G. Satorras, Z. Akata, and M. Welling, "Combining generative and discriminative models for hybrid inference," *Advances in Neural Information Processing Systems*, 2019.

[16] A. Klushyn *et al.*, "Latent matters: Learning deep state-space models," *Advances in Neural Information Processing Systems*, 2021.

[17] T. Imbiriba *et al.*, "Augmented physics-based machine learning for navigation and tracking," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 3, pp. 2692–2704, 2024.

[18] A. Chiuso and G. Pillonetto, "System identification: A machine learning perspective," *Annu. Rev. Control. Robotics Auton.*, vol. 2, pp. 281–304, 2019.

[19] G. Revach *et al.*, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.

[20] ——, "RTSNet: Learning to smooth in partially known state-space models," *IEEE Trans. Signal Process.*, vol. 71, pp. 4441–4456, 2023.

[21] G. Choi *et al.*, "Split-KalmanNet: A robust model-based deep learning approach for state estimation," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12 326–12 331, 2023.

[22] I. Buchnik *et al.*, "Latent-KalmanNet: Learned Kalman filtering for tracking from high-dimensional signals," *IEEE Trans. Signal Process.*, vol. 72, pp. 352–367, 2023.

[23] J. Wang, X. Geng, and J. Xu, "Nonlinear Kalman filtering based on self-attention mechanism and lattice trajectory piecewise linear approximation," *arXiv preprint arXiv:2404.03915*, 2024.

[24] X. Ni, G. Revach, and N. Shlezinger, "Adaptive KalmanNet: Data-driven Kalman filter with fast adaptation," in *Proc. IEEE ICASSP*, 2024.

[25] S. Chen *et al.*, "MAML-KalmanNet: A neural network-assisted Kalman filter based on model-agnostic meta-learning," *IEEE Trans. Signal Process.*, vol. 73, pp. 988–1003, 2025.

[26] N. Shlezinger and T. Routtenberg, "Discriminative and generative learning for linear estimation of random signals," *IEEE Signal Process. Mag.*, vol. 40, no. 6, pp. 75–82, 2023.

[27] N. Shlezinger, Y. C. Eldar, and S. P. Boyd, "Model-based deep learning: On the intersection of deep learning and optimization," *IEEE Access*, vol. 10, pp. 115 384–115 398, 2022.

[28] H. E. Rauch, F. Tung, and C. T. Striebel, "Maximum likelihood estimates of linear dynamic systems," *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, 1965.

[29] L. H. Kriouar, "Learning the kalman filter parameters: An expectation–maximization approach," Master's Thesis, University of Groningen, 2023.

[30] Y. Dahan *et al.*, "Bayesian KalmanNet: Quantifying uncertainty in deep learning augmented Kalman filter," *IEEE Trans. Signal Process.*, vol. 73, pp. 2558–2573, 2025.